

Rate Limiter

Сравнение алгоритмов ограничения частоты запросов

Проблематика и задачи

Rate Limiter — механизм контроля интенсивности входящего трафика.

Зачем он нужен в высоконагруженных системах?

- Защита инфраструктуры от перегрузок и DDoS-атак.
- Справедливое распределение ресурсов между клиентами.
- Контроль квот для публичных API.

Цель проекта: Разработать библиотеку на C++23, которая решает эти задачи максимально быстро, потокобезопасно и с использованием TTL механизмов.

Алгоритм: Token Bucket (Корзина токенов)

Алгоритм накапливает “токены” со скоростью r до максимума b . Каждый запрос стоит некоторое натуральное число токенов.

Плюсы:

- $O(1)$ по памяти (храним только время и счетчик).
- Отлично справляется с пиковыми нагрузками (bursts) — накопленные токены позволяют мгновенно пропустить пачку легитимных запросов.

Минусы:

- При неправильно заданном b внезапный всплеск может “пробить” защиту и перегрузить бэкенд.

Алгоритм: Leaking Bucket (Протекающее ведро)

Работает как строгая FIFO-очередь с константной скоростью обработки r .
Если очередь переполнена, новые запросы отбрасываются.

Плюсы:

- Идеально сглаживает трафик.
- Выдает стабильный, предсказуемый поток запросов на сервер.

Минусы:

- При резком пике очередь забивается старыми запросами.
- Нет гарантий по времени задержки (latency зависит от позиции в очереди).

Алгоритм: Sliding Window Log (Скользящий журнал)

Хранит временные метки (timestamps) **каждого** запроса в скользящем окне W .

Плюсы:

- Идеальная математическая точность.
- Нет “слепых зон” и скачков на границах временных интервалов.

Минусы:

- $O(N)$ потребление памяти (на каждого клиента нужен свой контейнер/куча).
- Дорогой пересчет при каждом новом запросе — падение пропускной способности при жесткой конкуренции.

Теория: Паттерны трафика

Для всесторонней оценки алгоритмов система тестировалась на пяти профилях нагрузки, отражающих реальные сценарии:

- **Uniform:** Равномерный константный трафик (например, телеметрия IoT).
- **Burst:** Резкие пики на фоне низкой базовой нагрузки (старт распродаж, push-уведомления).
- **Diurnal:** Плавные суточные волнообразные циклы активности (социальные сети).
- **Sparse + Burst:** Долгие простои с редкими, но мощными всплесками (batch-обработка).
- **Zombie:** Паразитный несинхронизированный трафик (активность вредоносных ботов, парсинг).

IoT — Internet of Things (Интернет вещей)

Теория: Параметры системы

Для настройки экспериментов и эмуляции реальной нагрузки задаются параметры:

- **Лимиты:** rate (целевой RPS) и burst_capacity (емкость бакета).
- **TTL ключей:** Время жизни неактивного клиента до удаления сборщиком мусора.
- **Длительность:** Время прогона одного теста для прогрева кэшей и стабилизации.
- **Конкурентность:** Количество параллельных потоков-клиентов.
- **Итерации:** Число повторных запусков для статистической значимости.

Теория: Метрики оценки

Сравнение эффективности алгоритмов проводится по шести ключевым метрикам:

- **Accuracy:** Точность ограничения частоты запросов (в %).
- **Throughput:** Максимальный чистый RPS, который выдерживает алгоритм.
- **Latency:** Задержки p50 (базовая скорость) и p99 (худшие случаи при локах).
- **Memory usage:** Потребление оперативной памяти на один ключ.
- **Burst handling:** Скорость и корректность обработки лавинообразного трафика.
- **Eviction count:** Число очищенных устаревших ключей (контроль памяти).

Архитектура: Базовые интерфейсы

Система разделена на синхронную и асинхронную обработку трафика:

IRateLimiter (Синхронный лимитер)

- Мгновенное решение: пропустить или отклонить (`Access(key)`).
- Состояние каждого ключа строго изолировано (квота одного не влияет на другого).

IRateShaper (Асинхронный шейпер)

- Формирование трафика (Traffic Shaping) через очередь.
- Возвращает `std::future<bool>`, который переходит в состояние `true`, когда запрос пропускается фоновым потоком.
- Реализация: `LeakyBucketShaper`.

Управление памятью и TTL

Для очистки базы данных от устаревших ключей и более эффективного использования памяти внедрен механизм **Time-to-Live (TTL)**.

Архитектура хранения:

- `MutexStorage`: потокобезопасная `std::unordered_map`.
- `StoredData`: оболочка над алгоритмом. Использует идиому **C RTP**, встраивая проверки `IsExpired()` прямо в `Access()` без накладных расходов на виртуальные вызовы.

Стратегии очистки устаревших ключей:

1. **Ленивая (Lazy)**: проверка и удаление в момент обращения к ключу.
2. **Фоновая (Eager)**: независимый поток (`RunCleaner`), сканирующий хранилище.

Потокобезопасность: Двухуровневая блокировка

Параллельные запросы к одному ключу вызывают **состояние гонки (data race)**, которое не решить одними атомиками.

Мы применили двухуровневую систему синхронизации:

1. **std::shared_mutex**: Защищает саму хеш-таблицу. `shared_lock` для обновления существующих ключей, `unique_lock` для вставки новых.
2. **Локальный std::mutex**: Опциональный мьютекс на каждый ключ (хранится внутри `MutexStorage`), гарантирующий последовательное выполнение логики `Access()`.

Итог: параллельные запросы к "user_A" корректно синхронизируются, не блокируя при этом запросы к "user_B".

Масштабирование: ShardedWrapper

При экстремальном RPS мьютекс таблицы становится узким горлышком.

Решение: Горизонтальное шардирование. Система разбивается на N независимых корзин. Маршрутизация ключей идет по формуле $\text{hash}(\text{key}) \% N$.

Оптимизация кэша (Борьба с False Sharing): Если независимые шарды лежат рядом в памяти, ядра процессора инвалидируют кэш друг друга. Мы выровняли сами шарды в памяти (используя LCM от размера кэш-линии и `alignof`), аппаратно разнеся их данные. Это полностью исключает ложное разделение.

LCM — Least Common Multiple (наименьшее общее кратное)

Условия экспериментов

Общие параметры бенчмарков:

- **Потоки:** 8 рабочих потоков (эмуляция жесткой конкуренции).
- **Длительность:** Минимум 10 секунд на итерацию, 3 повторения.
- **Метрики:** p50 и p99 latency вызова `Access()`.

Исследуемые паттерны:

1. **Uniform:** Равномерный трафик (1000 RPS).
2. **Zombie:** Паразитный фон с частой ротацией ключей (5000 RPS).
3. **Burst:** Редкие пиковые всплески.

Гипотеза 1: Конкурентная нагрузка (Uniform)

Суть: ShardedWinLimiter даёт меньший p99, чем MutexWinLimiter.

Реализация	p50, нс	p99, нс	Δ p50	Δ p99
MutexWinLimiter	1 833	60 600	—	—
ShardedWin (compact)	1 733	51 500	−5.5%	−15.0%
ShardedWin (aligned)	1 600	39 467	−12.7%	−34.9%

Вывод: Выравнивание шардов по кэш-линии (Aligned) снижает p99 на 35%.

Гипотеза **подтверждена**.

Гипотеза 2: Паразитный трафик (Zombie)

Суть: Sliding Window хуже Token Bucket по latency при сильной ротации ключей.

Реализация	p50, нс	p99, нс	Δ p50	Δ p99
Token Bucket	1 100	107 533	—	—
Sliding Window	2 700	194 367	+145%	+81%

Вывод: p50 выше в 2.5 раза. Token Bucket хранит $O(1)$ состояние, а Window вынужден постоянно аллоцировать память под новые метки времени. Гипотеза **частично подтверждена** (по latency подтверждена, память не замерялась).

Гипотеза 3: Всплески (Burst)

Суть: Token Bucket быстрее пропускает легальный всплеск.

Реализация	p50, нс	p99, нс	Δ p50	Δ p99
Token Bucket	1 167	33 267	—	—
Sliding Window	1 500	56 667	+29%	+70%

Вывод: На спайке Bucket списывает токен за $O(1)$, тогда как Window делает пересчет журнала на каждый запрос в пачке.

Гипотеза **подтверждена**.

Сводка результатов

№	Гипотеза	Итог
1	Sharded p99 < Mutex p99 @ 1000 RPS	подтверждена
2	Window хуже Token на Zombie	частично подтверждена
3	Token быстрее на Burst	подтверждена